01_channel_detection

October 8, 2026

1 Channel detection, and the control test

Aiming to use satellite imagery to find a river that no longer exists. A buried channel is filled with sand, which holds water better than the silt around it, so the ground above stays damper into the dry season. That shows up as darker soil in shortwave infrared and greener vegetation. The signal is weak in any one image, so we stack several years and look for what recurs.

Haven't trained any models here yet, just looking at the different principal components in pixels near the water to see if they are more similar to each other than they are to pixels of the ground.

[1]:
```
import sys, pathlib
sys.path.insert(0, str(pathlib.Path.cwd().parent / 'src'))

import numpy as np
import matplotlib.pyplot as plt
from scipy import ndimage

ROOT = pathlib.Path.cwd().parent
print('project root:', ROOT)
```

```
project root: /Users/vivek/Library/Mobile Documents/com~apple~CloudDocs/A
School/Harvard/Thesis/Code
```

1.1 1. Build seasonal composites

Sentinel-2 images are read from a public catalogue. Each covers 110 km across, so we request only the 20 km rectangle we need rather than the whole file. Every scene carries a layer labelling cloud and shadow, and those pixels are discarded.

For each band we then take the middle value across all the scenes in a season. The middle rather than the average, because the cloud mask always misses a few bright edges and one of those would drag an average upward.

Dry and wet seasons are kept apart. A buried channel shows itself by staying damp, so the contrast only exists once the ground around it has dried.

[2]:
```
from sat import stack

REGIONS = {
```

```
    'indus':   dict(bbox=[67.45, 24.70, 67.70, 24.92], label='Indus delta,␣
 ↪Banbhore'),
    'periyar': dict(bbox=[76.15, 10.10, 76.40, 10.32], label='Periyar delta,␣
 ↪Pattanam'),
}

composites = {}
for name, cfg in REGIONS.items():
    composites[name] = {
        season: stack.cached_composite(name, cfg['bbox'], '2022-01-01',␣
 ↪'2024-12-31', season)[0]
        for season in ('dry', 'wet')
    }
    shp = composites[name]['dry']['red'].shape
    print(f"{name}: {shp[0]} x {shp[1]} pixels at 10 m")
```

```
indus: 2463 x 2553 pixels at 10 m
periyar: 2444 x 2748 pixels at 10 m
```

1.2 2. Score every pixel

Each pixel is now 24 numbers: eight bands and four derived ratios, twice over for the two seasons. Principal components finds the directions along which pixels differ most.

The first component is reliably overall brightness and carries nothing about channels, so it is dropped. The score is the combined magnitude of the next three.

[3]:
```
from sat import detect

scores = {}
for name in REGIONS:
    score, comps, evr, names, pick, alt = detect.
 ↪spectral_score(composites[name])
    scores[name] = score
    print(f"{name}: variance explained by the first five components "
          f"{', '.join(f'{v:.2f}' for v in evr[:5])}")
```

```
indus: variance explained by the first five components 0.57, 0.25, 0.08, 0.04,
0.02
periyar: variance explained by the first five components 0.46, 0.33, 0.07, 0.05,
0.02
```

1.3 3. Define the control

Water is identified by a physical property as it absorbs near-infrared almost completely, so `(green - NIR) / (green + NIR)` is positive over water and negative over land.

The test zone is a ring of dry land two to twelve pixels outside the waterline. The water itself is excluded deliberately, because otherwise the detector could score well simply by rediscovering water, and what we actually want is land that a channel has affected.

The comparison group is land at least 600 m from any water.

[4]:
```
from sat import control

zones = {}
for name in REGIONS:
    med = composites[name]['dry']
    test_zone, water = control.riparian_control(med)
    far_from_water = (~ndimage.binary_dilation(water, iterations=60)
                      & np.isfinite(scores[name]) & ~test_zone)
    zones[name] = dict(test=test_zone, far=far_from_water, water=water)
    print(f"{name}: {test_zone.sum():,} pixels beside water, "
          f"{far_from_water.sum():,} well away from it")
```

```
indus: 415,077 pixels beside water, 4,243,568 well away from it
periyar: 550,661 pixels beside water, 2,808,383 well away from it
```

1.4 4. The test

Basically pulling from two bags of numbers: the scores of pixels beside water, and the scores of pixels far from it. Computing the AUC to see whether pixels beside water score systematically higher than pixels away from it.

[5]:
```
rng = np.random.default_rng(0)
THRESHOLD = 0.75

def auc(score, group_a, group_b, n=40000):
    a = score[group_a & np.isfinite(score)]
    b = score[group_b]
    a = rng.choice(a, min(n, a.size), replace=False)
    b = rng.choice(b, min(n, b.size), replace=False)
    ranks = np.concatenate([a, b]).argsort().argsort() + 1.0
    return (ranks[:len(a)].sum() - len(a) * (len(a) + 1) / 2) / (len(a) *␣
 ↪len(b)), a, b

results = {}
for name in REGIONS:
    value, a, b = auc(scores[name], zones[name]['test'], zones[name]['far'])
    results[name] = dict(auc=value, a=a, b=b)
    print(f"{name:9s} AUC = {value:.3f}   {'passes' if value >= THRESHOLD else␣
 ↪'fails'}")
```

```
indus     AUC = 0.814   passes
periyar   AUC = 0.578   fails
```

1.5 5. The figure

True colour, the score, and the two distributions side by side. If the detector works the blue distribution sits to the right of the grey one; if it does not, they pile on top of each other.

[6]:
```
def stretch(a, lo=2, hi=98):
    v = a[np.isfinite(a)]
    p0, p1 = np.percentile(v, [lo, hi])
    return np.clip((a - p0) / (p1 - p0 + 1e-9), 0, 1)

fig, axes = plt.subplots(2, 3, figsize=(16, 10.6), gridspec_kw={'width_ratios':␣
 ↪[1, 1, 0.92]})

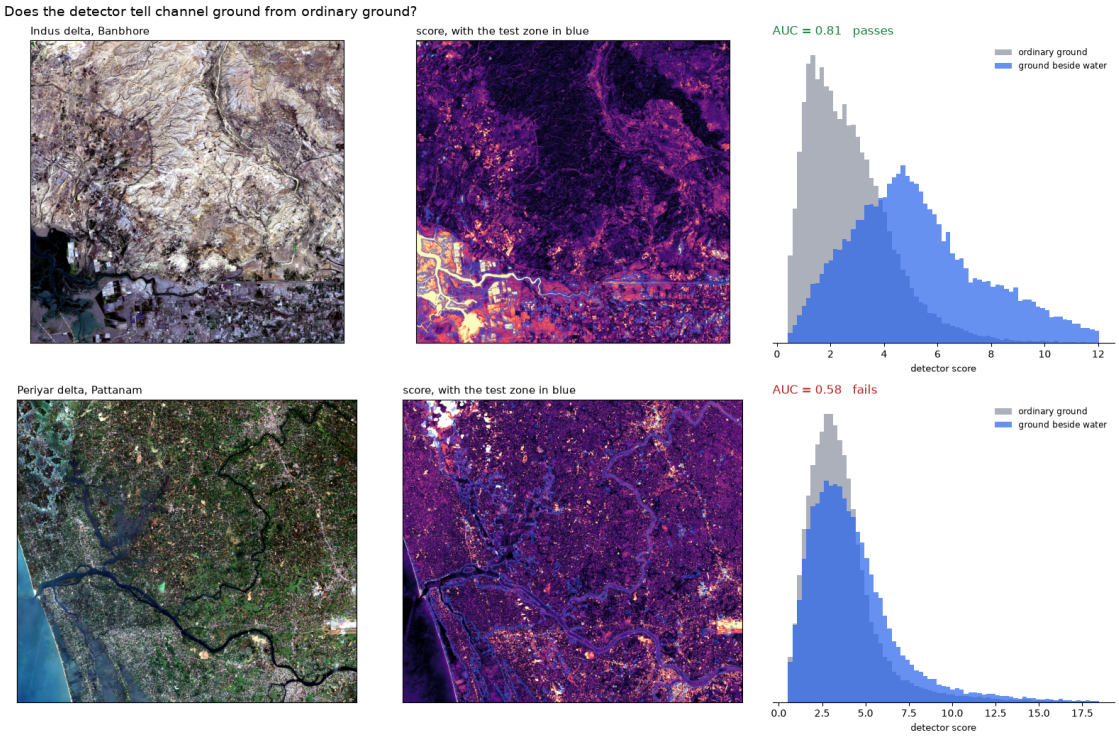
for row, (name, cfg) in enumerate(REGIONS.items()):
    med = composites[name]['dry']
    axes[row, 0].imshow(np.dstack([stretch(med['red']), stretch(med['green']),␣
 ↪stretch(med['blue'])]))
    axes[row, 0].set_title(cfg['label'], fontsize=10.5, loc='left')

    axes[row, 1].imshow(stretch(scores[name]), cmap='magma')
    overlay = np.zeros((*zones[name]['test'].shape, 4))
    overlay[zones[name]['test']] = [0.15, 0.39, 0.92, 0.55]
    axes[row, 1].imshow(overlay)
    axes[row, 1].set_title('score, with the test zone in blue', fontsize=10.5,␣
 ↪loc='left')

    r = results[name]
    lo, hi = np.percentile(np.concatenate([r['a'], r['b']]), [0.5, 99.5])
    bins = np.linspace(lo, hi, 70)
    ax = axes[row, 2]
    ax.hist(r['b'], bins=bins, color='#9ca3af', alpha=.85, density=True,␣
 ↪label='ordinary ground')
    ax.hist(r['a'], bins=bins, color='#2563eb', alpha=.70, density=True,␣
 ↪label='ground beside water')
    passed = r['auc'] >= THRESHOLD
    ax.set_title(f"AUC = {r['auc']:.2f}   {'passes' if passed else 'fails'}",
                 fontsize=12, loc='left', color='#15803d' if passed else␣
 ↪'#b91c1c')
    ax.set_xlabel('detector score', fontsize=9)
    ax.set_yticks([]); ax.legend(fontsize=8.5, frameon=False)
    ax.spines[['top', 'right', 'left']].set_visible(False)

    for a_ in axes[row, :2]:
        a_.set_xticks([]); a_.set_yticks([])

fig.suptitle('Does the detector tell channel ground from ordinary ground?',
             fontsize=14, x=0.012, ha='left')
fig.tight_layout()
plt.show()
```



1.6 What this does and does not establish

It establishes that the detector responds to ground beside a channel, strongly in the arid Indus and not at all in Kerala, where the landscape is cut through with paddy bunds and canals that produce the same linear damp signatures.

Has not yet established that the detector finds a channel which is now completely dry, will need to continue to work on that.